Towards Robust and Efficient Continual Language Learning

Adam Fisch[1,*] **Amal Rannen-Triki**[2] **Razvan Pascanu**[2] **Jörg Bornschein**[2]

Angeliki Lazaridou[2] **Elena Gribovskaya**[2] **Marc'Aurelio Ranzato**[2]

[1]MIT CSAIL [2]Google DeepMind

arXiv:2307.05741v1 [cs.CL] 11 Jul 2023

Abstract

As the application space of language models continues to evolve, a natural question to ask is how we can quickly adapt models to new tasks. We approach this classic question from a continual learning perspective, in which we aim to continue fine-tuning models trained on past tasks on new tasks, with the goal of "transferring" relevant knowledge. However, this strategy also runs the risk of doing more harm than good, i.e., negative transfer. In this paper, we construct a new benchmark of task sequences that target different possible transfer scenarios one might face, such as a sequence of tasks with high potential of positive transfer, high potential for negative transfer, no expected effect, or a mixture of each. An ideal learner should be able to maximally exploit information from all tasks that have any potential for positive transfer, while also avoiding the negative effects of any distracting tasks that may confuse it. We then propose a simple, yet effective, learner that satisfies many of our desiderata simply by leveraging a selective strategy for initializing new models from past task checkpoints. Still, limitations remain, and we hope this benchmark can help the community to further build and analyze such learners.

1 Introduction

Recent advances in large, pre-trained language models (LMs) have re-defined the ways practitioners approach and solve tasks in language understanding and generation (Devlin et al., 2019; Raffel et al., 2020; Brown et al., 2020; Rae et al., 2021; Hoffmann et al., 2022; Chowdhery et al., 2022, *etc*). Autoregressive language modeling removes the need for bespoke neural architectures, and provides a flexible framework for expressing diverse and complex tasks with unified input and output formats. At scale, LMs have achieved state-of-the-art performance across nearly every widely-used natural language processing (NLP) benchmark, and have had widespread impact in popular applications such as `ChatGPT` (Schulman et al., 2023).

Though much interest has focused on the few-shot and zero-shot reasoning abilities of very large LMs, an effective approach to solving targeted NLP tasks is still to take the parameters of a pre-trained model, and fine-tune them on data from the new task (Raffel et al., 2020; Gao et al., 2021; Wei et al., 2022). Here there is a similar trend: performance generally improves as the LM grows. Unfortunately this also results in significant computational costs during fine-tuning, even if the number of updates required is ultimately less than would be required if training the model from scratch. Furthermore, fine-tuning is typically performed independently for each new task, and ignores any other tasks the LM might have previously been applied to. This not only leads to an accrual of computational cost over all tasks, but also fails to *share* acquired knowledge across tasks. In this work, we revisit fine-tuning efficiency from a continual learning perspective, motivated by the following question:

Question 1. *Suppose that we have already solved a set of n previous tasks, $\{t_1, \ldots, t_n\}$. Can we leverage any of the information gained from these tasks to solve the next task t_{n+1} more efficiently?*

Specifically, we study the setting where each of the previous n tasks is associated with a substantial number of training examples (e.g., several thousand). This setting is common, but not well addressed by few-shot prompting. Our conjecture, which has also previously found empirical support in various related NLP settings (Phang et al., 2019; Poth et al., 2021; Choshen et al., 2022, *inter alia*), is that the standard pre-trained model—which wide-spread wisdom uses as the default initialization for any fine-tuning task—might not in-fact be the best checkpoint to use. Rather, models derived from one (or a combination) of the previous tasks

*Work done while an intern at Google DeepMind.

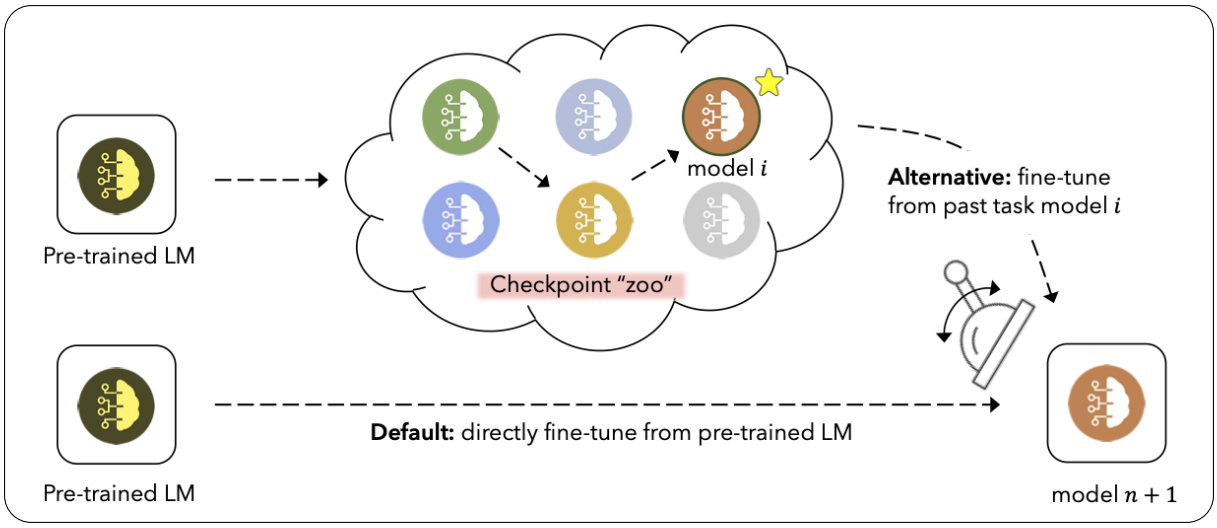

Figure 1: An illustration of our continual learning framework. When training the $(n+1)$th model we choose between initializing from the default pre-trained language model and a previously fine-tuned model. This is repeated for each new task, and models within the zoo may build off each other to create a chain of fine-tuned models. Our motivation is to make fine-tuning more efficient, while also being robust to the composition of previous tasks.

might work even better as a starting point. This assumes that "knowledge transfer", or a form thereof, is accomplished via parameter initialization.

We measure performance by how quickly our learning algorithm can produce good models for new tasks. Specifically, how much computational budget do we need to produce a model with some desired performance level? Or, alternatively, for a given computational budget, what is the best performance that we can achieve? Put in the context of past work on continual learning (Wołczyk et al., 2021; Veniat et al., 2021; Bornschein et al., 2022), we are focused on forward transfer, which we define as having a faster "rate of learning" on a new task—relative to our baseline strategy of independent fine-tuning from the original pre-trained LM parameters. Naturally, how well one can hope to do depends not only on the algorithm that is used, but also on the relationships between the new task and previous ones. We conduct a large-scale analysis of pairwise interactions across 55 popular and publicly available (English) language tasks using a T5 LM (Raffel et al., 2020). Here we first fine-tune a T5 LM on task **A**, and then proceed to fine-tune it on task **B**. This indeed results in a fairly tumultuous transfer landscape: in some cases pre-training on **A** first can result in faster adaptation to task **B**, but in other cases it can be quite detrimental. How can we expect which situation we may encounter, especially when faced with not just one, but many previous tasks and task combinations to transfer from?

We argue that practical, efficient continual learning demands algorithms that are robust to the inevitable variations in the composition of the previous n tasks. To this end, guided by our pairwise matrix of task interactions, we construct a challenging benchmark of multiple task sequences $(t_1, t_2, \ldots)$ that target different possible scenarios one might face, such as a sequence of tasks with high potential positive transfer, high potential for negative transfer, no expected effect, or a mixture of each. An ideal continual learner should be able to exploit information from all tasks that have any potential for positive transfer, while also avoiding the harmful effects of any "distractor" tasks that may confuse it (and result in negative transfer).

As a first step, we propose a simple method that manages to satisfy many of our desiderata. Concretely, we learn a checkpoint selection model that, given some representation of the current task t_{n+1} and for all previously seen tasks $(t_1, \ldots, t_n)$, predicts which previously saved checkpoint is the best checkpoint to initialize from—including the default option "t_0", which is simply the pre-trained model that a standard fine-tuning approach would start from. We demonstrate that training a lightweight gradient boosted decision tree (Friedman, 2001) on top of (fast and easy to derive) features of each task over a small collection of held-out task pairs with different positive, negative, or neutral pairwise transfer relationships can result in good selection performance on new tasks: particularly when there exist *harmful* past tasks that are best to be ignored.

In short, the core idea and contribution of this work can be summarized quite simply:

1. We motivate and explore continual learning for efficient LM fine-tuning and forward transfer;
2. To support this direction, we present a large-scale analysis of pairwise task transfer interactions, and a new benchmark of task sequences that capture diverse potential transfer profiles;
3. Finally, we give a simple but effective method for checkpoint selection and model initialization that helps enable more robust forward transfer.

2 Related work

Forward transfer. This work builds on a large body of recent work that seeks to improve the efficiency of training modern language models through forward transfer (via parameter initialization). In particular, leveraging auxiliary task data to improve target task performance has been a very active area of research years over the past few years (Luong et al., 2016; Bingel and Søgaard, 2017; Phang et al., 2019; Wang et al., 2019; Gururangan et al., 2020; Pruksachatkun et al., 2020; Vu et al., 2020; Chang and Lu, 2021; Aribandi et al., 2022). Our work falls under the category of *intermediate fine-tuning*, where a model is first trained on some auxiliary task $\mathbf{A}$ before being transferred to a target task $\mathbf{B}$. This paradigm has been well-analyzed in the pair-wise setting (i.e., $\mathbf{A} \rightarrow \mathbf{B}$ only), and multiple past studies have given empirical guidelines on how to select optimal transfer pairs (Ruder and Plank, 2017; Deshpande et al., 2021; Poth et al., 2021; Huang et al., 2022; Choshen et al., 2022; You et al., 2021, 2022). Here, we extend intermediate task training in a pair-wise fashion to training over full sequences of intermediate (and continually learned) tasks.

Continual learning. The key focus of this work is on continual learning for efficient language learning. Over the past decade, continual learning research has received significant interest within the wider machine learning community; see, e.g., Parisi et al. (2019) for a review. Methodology-wise, existing work on continual learning can be approximately categorized into (a) replay-based (de Masson d'Autume et al., 2019; Scialom et al., 2022), (b) regularization-based (Kirkpatrick et al., 2017; Chaudhry et al., 2019; Qin et al., 2022; Ke et al., 2023), or (c) architecture-based (Carlson et al., 2010; Veniat et al., 2021; Douillard et al., 2022; Razdaibiedina et al., 2023; Qin et al., 2023) approaches. Many of these methods are motivated both by parameter-efficient forward transfer, as well as resistance to catastrophic forgetting. In contrast, similar to Bornschein et al. (2022), we are only interested in *training* efficiency on the new task—without worrying about how performance might suffer on previous tasks—and focus only on different model initialization strategies for simplicity.[1] Our benchmark of transfer sequences also adds to a growing collection of continual learning datasets and analysis techniques for NLP (Lazaridou et al., 2021; Livska et al., 2022; Jang et al., 2022; Wu et al., 2022), with an attention towards sequences of a particular challenging structure that stress test for robustness to negative transfer.

Efficient training for NLP. Finally, our work is also more broadly related to efficient training in language models (Mattson et al., 2020; Geiping and Goldstein, 2022; Menghani, 2023), which also includes efficiency in terms of parameter reuse and stored model size (Houlsby et al., 2019; Li and Liang, 2021; He et al., 2022; Hu et al., 2022; Lei et al., 2023). While we do not consider these forms of efficiency in this work, they can be complementary to the form of training efficiency that we do concentrate on. Some of the metrics and analysis we propose may also be of independent interest.

3 Problem formulation

Let $\mathrm{LM}_\theta \colon \mathcal{X} \rightarrow \mathcal{Y}$ be our parametric language model, which generates natural language responses $y \in \mathcal{Y}$ given a prompt $x \in \mathcal{X}$. All of our experiments use the pre-trained T5 base model of Raffel et al. (2020), specifically the version adapted for language modeling by Lester et al. (2021).

3.1 Fine-tuning efficiency

During fine-tuning, the model parameters vary as a function of the number of update steps $s \in \mathbb{N}$ that have been taken, denoted as $\theta(s)$. We quantify the time-dependent performance of our updating language model, $\mathrm{LM}_{\theta(s)}$, by its **best** (i.e., minimum) loss achieved within a budget of B update steps:

$$\mathrm{Perf}(B) := \min \Big\{ \underbrace{\mathbb{E}_{X,Y}\left[\ell(\mathrm{LM}_{\theta(s)}(X), Y)\right]}_{\text{average loss after step } s} : s \leq B \Big\}, \tag{1}$$

[1] This is motivated by our simplifying assumptions that we (a) know what task we are trying to solve at each point in time, and (b) can checkpoint and load past models when necessary.

Algorithm 1 Sequential fine-tuning

$\theta_0 \leftarrow$ Pre-trained model
for $t_i \in t_1, t_2, \ldots$ **do**
  # Initialize starting point from a previous model.
  $\theta_i(0) \leftarrow \text{SELECT}(\theta_0, \ldots, \theta_{i-1})$
  # Update current model using task data (X_i, Y_i).
  for $s \in 1, 2, \ldots, B$ **do**
    $\theta_i(s) \leftarrow \text{UPDATE}(\text{LM}_{\theta_i(s-1)}, X_i, Y_i)$
  # Keep the best model from the past B steps.
  $\theta_i \leftarrow \underset{s \leq B}{\text{argmin}}\ \mathbb{E}_{X_i,Y_i}[\ell(\text{LM}_{\theta_i(s)}(X_i), Y_i)]$

where $\ell\colon \mathcal{Y} \times \mathcal{Y} \to \mathbb{R}$ is an arbitrary loss metric.

To reduce complexity and confounders between implementations, we choose to use the same model architecture, batch size, and learning rates in all of our experiments. The consequences of this are that (1) the number of update steps is directly proportional to the total training cost of the model, and (2) achieving better $\text{Perf}(B)$ simply reduces to finding a better *initialization* for our model, i.e., a choice of $\theta(0)$ that gives rise to efficient trajectories $\theta(s)$.[2]

Finally, as an aggregate measure of $\text{Perf}(B)$ across budgets B, we evaluate the area under the performance curve as a function of log updates, up to a maximum number of updates B_{max}, i.e.,

$$\text{PerfAUC}(B_{\text{max}}) := \int_0^{\log B_{\text{max}}} \text{Perf}(e^b) db. \quad (2)$$

$\text{PerfAUC}(B_{\text{max}})$ will be our primary metric for comparing methods, where we set $B_{\text{max}} = 10k$, which is empirically the point for which the majority of models for our tasks have (nearly) converged. More specifically, we will be interested in measuring the *relative* efficiency of continual learning methods compared to the baseline of independent fine-tuning (where we always start from the same general pre-trained model for each task). Inspired by the relative forward transfer metrics of Wołczyk et al. (2021), we compute this relative score as

$$\frac{\text{PerfAUC}(B_{\text{max}})_{\text{ind}} - \text{PerfAUC}(B_{\text{max}})_{\text{m}}}{\text{PerfAUC}(B_{\text{max}})_{\text{ind}} - L}, \quad (3)$$

where $(\cdot)_{\text{m}}$, $(\cdot)_{\text{ind}}$ are the scores of the method and the baseline of independent fine-tuning, respectively, and L is the metric lower bound for $\text{PerfAUC}(B_{\text{max}})$ (e.g., $0\% \times \log B_{\text{max}}$ for error rate). Intuitively, this score measures the relative *improvement* in terms of how much the compared method reduces the performance gap to the oracle (i.e., perfect predictions starting from step 0).

[2]Note that an interesting direction for future work is to explore how the findings presented here generalize across different classes of models/learning algorithms (and if not, why).

Algorithm 2 "Naïve" sequential fine-tuning

function SELECT$(\theta_0, \ldots, \theta_{i-1})$
  # Return the most recently trained model.
  return θ_{i-1}

3.2 Sequential fine-tuning

As a starting point for the remainder of this paper, we now describe a very simple continual learning procedure for sequential fine-tuning on a stream of tasks $(t_1, t_2, \ldots)$, see also Algorithm 1. Beginning from an initial pre-trained language model LM_{θ_0}, we sequentially adapt models LM_{θ_i} one after the other by using the model learned on some previous task $t_{j<i}$ to initialize the model used on task t_i. Note that here we write θ_i to index the task parameters, and will use $\theta_i(s)$ to denote the task parameters as a function of the number of updates. As described earlier, we use the same model architecture, batch size, and learning rate for each task. The only setting that changes is the initialization. The "naïve" implementation of sequential fine-tuning is to simply select the most recent checkpoint, θ_{i-1}, see Algorithm 2. Of course, this procedure is not necessarily optimal, since the model parameters learned for a task $\mathbf{A}$ may not be a good initialization for another task $\mathbf{B}$. In the next section we present an analysis of when $\mathbf{A}$ does have potential to transfer well as a good initialization for $\mathbf{B}$, and when it does not.

4 Analyzing task transfer potential

To help guide our understanding of how well parameters learned from training on language task $\mathbf{A}$ perform when used as a starting point for training on a new language task $\mathbf{B}$, we conduct a large-scale analysis over various, diverse pairs of language tasks $(\mathbf{A}, \mathbf{B})$. Note that this is the *minimal* setting for which sequential fine-tuning can be applied.

4.1 Dataset collection

The tasks that we analyze are shown in Figure 2, and mainly follow those used by FLAN (Wei et al., 2022), but without translation (we do not use multilingual models here). We use the same (loosely defined) "task family" groupings as Wei et al. (2022) to help guide our analysis (below), but ultimately are interested in transfer between individual tasks. To identify "interesting" pairs $(\mathbf{A}, \mathbf{B})$ that

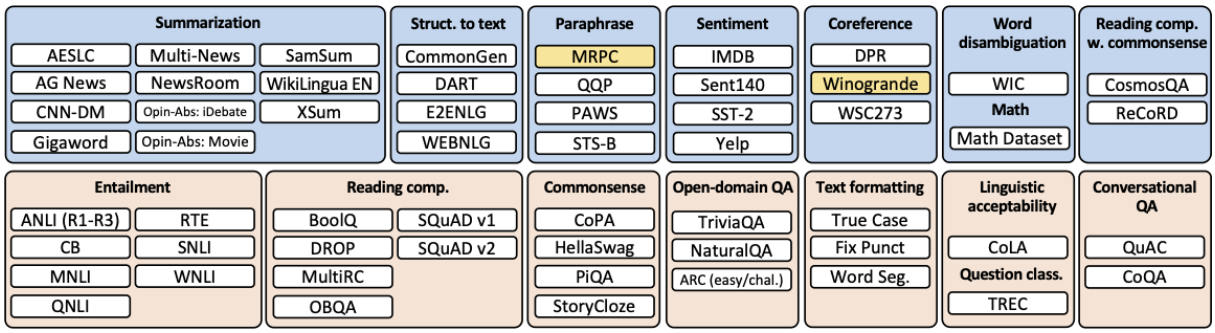

Figure 2: The collection of tasks used to create the sequential transfer benchmark used in this paper. Tasks are grouped into approximate "families", and families are further separated into training (top) and testing (bottom) splits. Highlighted training tasks are used for validation (i.e., task **C** when measuring transfer from **A** → **B** → **C**).

have either significantly negative or positive effects on each other, we use the following search strategy:

1. We evaluate all 16×16 task family pairs, where for a family pair $(\mathcal{F}_i, \mathcal{F}_j)$ we first train a model on a mixture of all tasks $t'_i \in \mathcal{F}_i$, and then use that model as the starting point for training the second model on a mixture of all tasks $t'_j \in \mathcal{F}_j$. Each model is trained on a balanced mixture of task data, and evaluated according to the average performance across tasks within each family.
2. For a pair $(\mathcal{F}_i, \mathcal{F}_j)$ the average performance after sequentially fine-tuning on $\mathcal{F}_j \rightarrow \mathcal{F}_j$ can either be better, worse, or approximately the same relative to training independently on $\mathcal{F}_j$. We use this signal as evidence that there may exist individual tasks $t'_i, t'_j \in \mathcal{F}_i \times \mathcal{F}_j$ with a similar trend.
3. For each family $\mathcal{F}_i$, we identify the top-K families $\mathcal{F}_j$ with the *best* average transfer to $\mathcal{F}_i$, as well as the worst-K families $\mathcal{F}_j$ with the *worst* average transfer to $\mathcal{F}_i$. K is set to 3. We then evaluate all individual task pairs in $\mathcal{F}_i \times \mathcal{F}_j \times \mathcal{F}_k$.

In total, we evaluate 1757 unique task pairs. Figure 3 plots the distribution of transfer results in terms of relative PerfAUC. Consistent with observations in prior work (Pruksachatkun et al., 2020; Poth et al., 2021), we can see that while on many tasks there is no marked effect due to sequential fine-tuning, there do exist a significant tails of both positive and negative transfer instances.[3]

4.2 Types of transfer profiles

Figure 4 gives a number of qualitative examples that exhibit some of the different types of

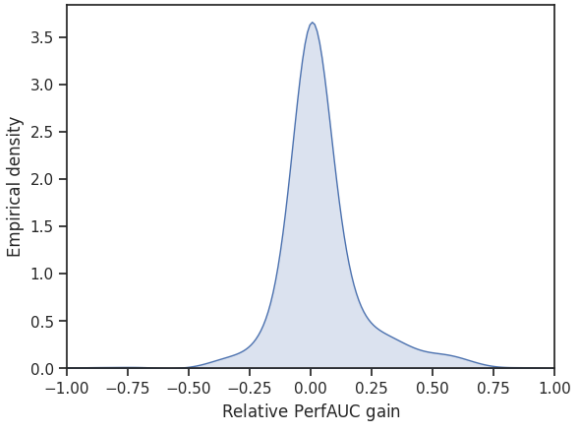

Figure 3: A density plot of the empirical distribution of the relative PerfAUC across the 1757 task pairs **A** → **B** that we (selectively) evaluate. All models are trained with "naïve" sequential fine-tuning, where we use the checkpoint of task **A** as a starting point for task **B**.

transfer profiles that arise. Some models exhibit strong transfer from the beginning: the 0-shot performance is good, and continues to improve. Good 0-shot performance, however, is not always a reliable indicator of future success: some models start out with better performance, but improve more slowly. Other pairs yield no tangible difference. There is also significant variation across tasks, with some tasks acting as fairly "universal donors" with positive transfer to most other tasks, while others mostly result in negative, or at best minimal positive, transfer. For example, in our experiments, 73% of models trained first on the STS-B dataset (Cer et al., 2017) had $> +5\%$ relative PerfAUC across evaluated target tasks. On the other hand, 70% of models trained first on the Math dataset (Saxton et al., 2019) had $< -5\%$ relative PerfAUC across evaluated target tasks.

Interpreting and understanding *why* these

[3]Note, however, that this distribution is also artificially *biased* towards the tails, due to our search and evaluation strategy. Nevertheless, it can still be inferred that substantial absolute numbers of both positive and negative instances exist.

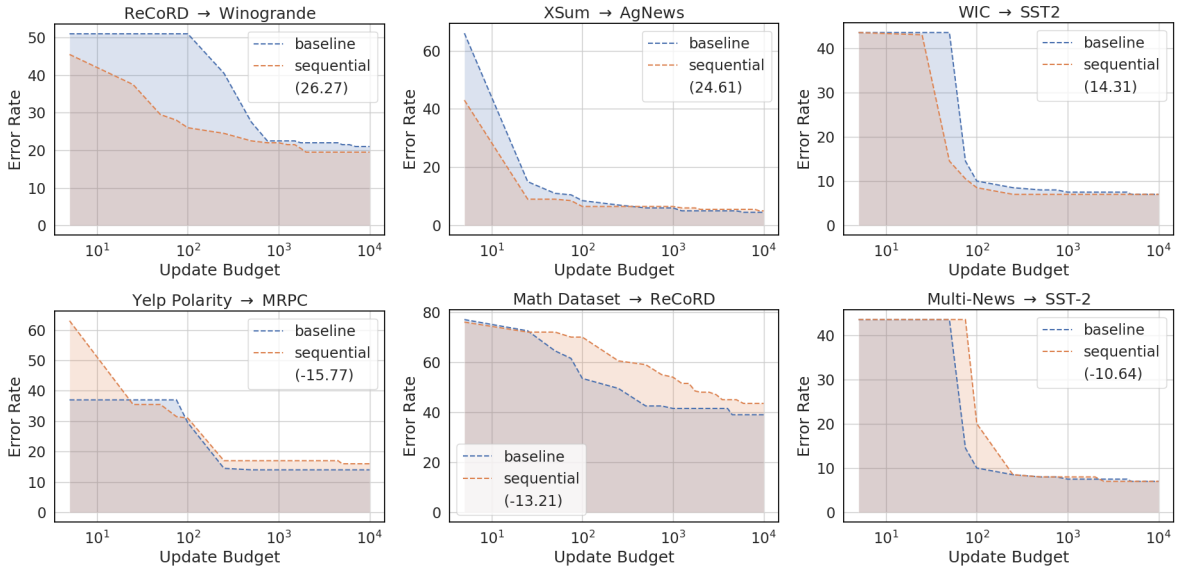

Figure 4: Example positive and negative pairwise transfer profiles $\mathbf{A} \rightarrow \mathbf{B}$, in which the lowest loss per update budget on task $\mathbf{B}$ is plotted. Blue is for the baseline of independent fine-tuning (pre-trained model $\rightarrow \mathbf{B}$), while orange is for "naïve" sequential fine-tuning (pre-trained model $\rightarrow \mathbf{A} \rightarrow \mathbf{B}$). Relative $\mathrm{PerfAUC}$ is included in each legend.

$\mathbf{A} \rightarrow \mathbf{C}$	$\mathbf{B} \rightarrow \mathbf{C}$	Desired $\mathbf{A} \rightarrow \mathbf{B} \rightarrow \mathbf{C}$
Positive	Positive	$\geq \max(\mathbf{A} \rightarrow \mathbf{C}, \mathbf{B} \rightarrow \mathbf{C})$
Positive	Negative	$\approx \mathbf{A} \rightarrow \mathbf{C}$
Positive	Neutral	$\approx \mathbf{A} \rightarrow \mathbf{C}$
Negative	Positive	$\approx \mathbf{B} \rightarrow \mathbf{C}$
Negative	Negative	$\approx \mathbf{C}$
Negative	Neutral	$\approx \mathbf{C}$
Neutral	Positive	$\approx \mathbf{B} \rightarrow \mathbf{C}$
Neutral	Negative	$\approx \mathbf{C}$

Table 1: Types of task triplets in our benchmark. The final column indicates the desired behavior on task $\mathbf{C}$ when using an "ideal" continual learning algorithm.

transfer curves form the way they do is tricky—and something we leave as an open problem for future research. Nevertheless, the existence of these empirical phenomena allows us to construct challenging sequences of tasks over which to perform continual learning. As previously discussed, an ideal learner should be able to exploit information from all tasks that have any potential for positive transfer (demonstrated by having a positive pairwise transfer result), while also avoiding the negative effects of any potentially harmful tasks that may confuse it (demonstrated by having a negative pairwise transfer result). An ideal learner should be agnostic to the mechanism that is responsible for the positive or negative transfer, which in many common situations (such as in many of the tasks presented here) may not be that well understood.

4.3 Constructing a diagnostic benchmark

We leverage the pairwise transfer results to construct a series of diverse, diagnostic task sequences. The format of these sequences is outlined in Table 1. We split the pairs of tasks into **positive**, **negative**, and **neutral** subsets based on the magnitude of their relative $\mathrm{PerfAUC}$ to the independent fine-tuning baseline (see Eq. 3). For positive/negative tasks, we attempt to account for variance in training (where the randomness is over the batch selection and ordering during SGD) by requiring that the mininum/maximum relative $\mathrm{PerfAUC}$ results across all random trials are above/below $+5\%/-5\%$, respectively (though occasional false positives exist across different runs). We then construct 8 different types of triplets $(\mathbf{A}, \mathbf{B}, \mathbf{C})$, where each of the preceding tasks $\mathbf{A}$ and $\mathbf{B}$ are mostly positive, negative, or neutral *pairwise* transfer sources for the target task $\mathbf{C}$ (i.e., $\mathbf{A} \rightarrow \mathbf{C}$ and $\mathbf{B} \rightarrow \mathbf{C}$, respectively). Note that we exclude the neutral/neutral case. For each configurations, we include multiple sets of triplets with different source and target tasks, and measure the median performance across task instances in all experiments. Specifically, on the test split of the benchmark, for each type of triplet (e.g., positive/positive) we include 4 distinct target tasks $\mathbf{C}$, each with 4 distinct preceding task pairs $(\mathbf{A}, \mathbf{B})$, for a total of 16 triplets per setting (and 128 triplets in total). Additional details are provided in Appendix A.

5 Learning a checkpoint selector

We now propose a straightforward, but effective, algorithm for robust forward transfer. Motivated by our analysis in Section 4, we consider a simplified version of Question 1 that we posed in Section 1:

Question 2. *Suppose that the set of previously solved tasks* $\{t_1, \ldots, t_n\}$ *contains a distinct set of tasks with trained models* θ_i *that act as good initializations for* t_{n+1}*, i.e.* $\mathcal{P} \subseteq \{t_1, \ldots, t_n\}$*. Given features* $\phi(t_i, t_{n+1}) \in \mathbb{R}^d$*, can we learn a discriminator* $\mathcal{D}\colon \mathbb{R}^d \to \{0, 1\}$ *to identify "positive" task candidates* $t_i \in \mathcal{P}$ *to leverage for learning* t_{n+1}*?*

Concretely, when training a new model for t_{n+1}, we seek to allow ourselves to select a previously fine-tuned model on some task $t_i \in \{t_1, \ldots, t_n\}$ to initialize from—if we think that it will lead to *positive* transfer. If multiple such tasks exist, then we select the most confident one, using confidence scores from some model $\mathcal{C}\colon \mathbb{R}^d \to [0, 1]$, which is typically the same underlying model as $\mathcal{D}$, but without a decision threshold. If we are not confident that any such task model exists, then we initialize from the default pre-trained language model. This process, which we call *selective* sequential fine-tuning, is illustrated in Algorithm 3, and is similar in spirit to prior work on checkpoint selection for transfer learning (see §2), with the caveat that we only select candidates that pass a decision threshold (see also §7 for a discussion on the potential importance of properly calibrating this threshold). This process is repeated for each new task, e.g., for a sequence $\mathbf{A} \to \mathbf{B} \to \mathbf{C}$, task $\mathbf{A}$ is initialized from the pre-trained model, task $\mathbf{B}$ is either initialized from the pre-trained model or the checkpoint for $\mathbf{A}$, and task $\mathbf{C}$ is either initialized from the pre-trained model or either of the checkpoints for $\mathbf{A}$ or $\mathbf{B}$. In general, there are 2^n possible paths (in terms of sequential initializations) to take from t_0 to task t_{n+1}.

We choose to instantiate $\mathcal{D}$ as a simple gradient boosted decision tree (GBDT) (Friedman, 2001) operating on several light-weight "meta" features, $\phi(t_i, t_j)$ of an input task pair. $\mathcal{C}$ is the pre-binarized decision function of the GBDT. The GBDT is trained over positive and negative pairs from the training split of our benchmark.[4] The features ϕ are fairly conventional (e.g., similar motivation can be found in the related approaches of Bingel and Søgaard (2017); Poth et al. (2021)). They include metadata (e.g., if any of the previous tasks are in the same family as t_j, the zero-shot and few-shot performance of model t_i on t_j, and a number of gradient-based similarity metrics comparing updates to t_i and t_j relative to a t_0 starting point. See Appendix B for more details. We binarize $\mathcal{D}$ by thresholding the GBDT confidence at 0.5 (i.e., we only consider a checkpoint to be a candidate for selection if it is judged by our model to be more likely than not to be a positive transfer pair).

[4]While we ignore this aspect in this work, note that this introduces a distributional shift at test time, since candidate models are themselves products of multiple iterations of this selection algorithm, rather than only pairwise transfer instances.

Algorithm 3 "Selective" sequential fine-tuning

function SELECT($\theta_0, \ldots, \theta_{i-1}$)
  # Estimate selection of positive transfer candidates
  # from the corresponding tasks $(t_1, \ldots, t_{i-1}.t_i)$,
  # $\mathcal{D}$ is a trained "positive transfer" discriminator.
  $\widehat{\mathcal{P}} \leftarrow \{t_j : \mathcal{D}(t_j, t_i) = 1, j < i\}$
  if $\widehat{\mathcal{P}} \neq \varnothing$ **then**
    # Pick the most confident candidate if any
    # exists, where $\mathcal{C}$ is a confidence measure.
    $j^* \leftarrow \text{argmax}_j \{\mathcal{C}(t_j, t_i) : t_j \in \widehat{\mathcal{P}}\}$
  else
    # Otherwise, default to the pre-trained model.
    $j^* \leftarrow 0$
  return θ_{j^*}

6 Results

We compare the behavior of naïve sequential fine-tuning, our selective sequential fine-tuning procedure, and an oracle checkpoint selection algorithm across the task sequences in our benchmark. Our results are given in Table 2. See also Appendix C. The oracle picks the best sequential fine-tuning path from t_0 to t_{n+1} in hindsight, and is used as an upper-bound to our selective model performance (as $n = 2$ in our experiments, this results in 4 total possible paths). We report the median relative PerfAUC result across all 16 triplets for each sequence type (e.g., for the triplet $\mathbf{A} \to \mathbf{B} \to \mathbf{C}$, where $\mathbf{A} \to \mathbf{B}$ results in positive pairwise transfer, while $\mathbf{B} \to \mathbf{C}$ results in negative pairwise transfer).

Forward transfer. Rows in Table 2 with green entries denote sequence types with potential for positive forward transfer. When both tasks $\mathbf{A}$ and $\mathbf{B}$ are positive intermediate tasks for task $\mathbf{C}$, continuing to fine-tune $\mathbf{A} \to \mathbf{B} \to \mathrm{C}$ generally also result in positive transfer—interestingly, often to a larger degree than either of only $\mathbf{A} \to \mathrm{C}$ or $\mathbf{B} \to \mathrm{C}$. When a positive intermediate task

A → **C** Pairwise	**B** → **C** Pairwise	**A** → **B** → **C** Naïve	Selective	Oracle
34.5	31.8	42.7	42.4	43.6
13.9	-17.0	-2.35	11.8	13.9
15.1	0.244	16.9	15.1	19.9
-17.0	13.7	12.0	11.8	15.5
-24.5	-19.5	-24.7	0.00	0.00
-17.0	-1.65	-8.34	0.00	0.00
-0.703	25.2	19.8	15.0	26.3
-1.88	-16.9	-12.6	0.00	0.00

Table 2: PerfAUC results on our benchmark sequences. Each row is the median of all 16 instances of that configuration (e.g., positive **A** → positive **B** → **C**. Green denotes intended "positive" pairwise transfer, red denotes "negative" pairwise transfer, while grey denotes "neutral" transfer (i.e., no substantial effect). Oracle is the best achievable result using any (possible) sequence of checkpoints from the initial pre-trained model to task **C**. A score of 0 means performing as well as a model that fine-tunes from the original pre-trained model, while positive/negative scores are improvements/degradations relative to that, which is the default used today.

is paired with a *negative* intermediate task (red entries), the performance of naïve sequential fine-tuning is sensitive to their ordering (and is better when the most recently trained task is a positive transfer pair). Our selective procedure, however, manages to leverage positive transfer where possible regardless of order—though it can significantly lag behind the oracle in certain cases.

Negative transfer. Rows in Table 2 with red entries denote sequence types with potential for negative transfer. Fortunately, unlike sequential transfer with positive transfer options, harmful effects from two negative, or negative and neutral, intermediate tasks **A** and **B** rarely compound (in fact, the negative effect can sometimes be attenuated). In the cases where there are no positive intermediate tasks to transfer from, our selective algorithm is successful in choosing the pre-trained model as a starting checkpoint (resulting in 0, but at least not *negative*, relative PerfAUC).

7 Limitations and challenges

While our work provides a starting point for testing robust and efficient continual learning, several limitations remain. Most significantly, our focus is restricted to T5 base models with simple optimization routines, and the only method of transfer that we test and explore is via parameter initialization, without considering space efficiency (i.e., in reducing the number of saved parameters across all tasks). Our selective checkpoint initialization strategy is therefore advantaged with respect to this particular setting. Additionally, our oracle is only evaluated for this strategy—other methods that use different knowledge transfer paradigms may do even better (Ermis et al., 2022; Qin and Joty, 2022; Razdaibiedina et al., 2023). We note that noise is also introduced though stochastic effects in SGD (e.g, learning rates, batch sizes), which introduces some confounding effects, especially when integrating over log updates (which biases PerfAUC towards early performance). This is more significant for some tasks than others. Finally, another challenge is that as the number of considered tasks grows, our selective classifier may become more prone to identifying a large number of false positives. Without using calibration techniques that account for multiple testing (e.g., Fisch et al., 2021), the selective classifier may choose poor checkpoints with increasingly high probability.

8 Conclusion

This paper develops a collection of task sequences with diverse transfer scenarios to test for efficient and robust continual learning on language tasks. Our benchmark targets different possible scenarios one might face: such as a sequence of tasks with high potential for positive transfer, negative transfer, no effect, or a mixture of each. As a first step, we proposed a selective algorithm for choosing past checkpoints to initialize from when considering each new task t_{n+1}. Limitations remain, and we hope this benchmark may help analyze and identify strong continual language learning algorithms.

References

Vamsi Aribandi, Yi Tay, Tal Schuster, Jinfeng Rao, Huaixiu Steven Zheng, Sanket Vaibhav Mehta, Honglei Zhuang, Vinh Q. Tran, Dara Bahri, Jianmo Ni, Jai Gupta, Kai Hui, Sebastian Ruder, and Donald Metzler. 2022. Ext5: Towards extreme multi-task scaling for transfer learning. In *International Conference on Learning Representations*.

Joachim Bingel and Anders Søgaard. 2017. Identifying beneficial task relations for multi-task learning in deep neural networks. In *Proceedings of the 15th Conference of the European Chapter of the Association for Computational Linguistics: Volume 2, Short Papers*, pages 164–169, Valencia, Spain. Association for Computational Linguistics.

Jorg Bornschein, Alexandre Galashov, Ross Hemsley, Amal Rannen-Triki, Yutian Chen, Arslan Chaudhry, Xu Owen He, Arthur Douillard, Massimo Caccia, Qixuang Feng, Jiajun Shen, Sylvestre-Alvise Rebuffi, Kitty Stacpoole, Diego de las Casas, Will Hawkins, Angeliki Lazaridou, Yee Whye Teh, Andrei A. Rusu, Razvan Pascanu, and Marc'Aurelio Ranzato. 2022. Nevis'22: A stream of 100 tasks sampled from 30 years of computer vision research.

Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel Ziegler, Jeffrey Wu, Clemens Winter, Chris Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language models are few-shot learners. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Andrew Carlson, Justin Betteridge, Bryan Kisiel, Burr Settles, Estevam R. Hruschka, and Tom M. Mitchell. 2010. Toward an architecture for never-ending language learning. In *Proceedings of the Twenty-Fourth AAAI Conference on Artificial Intelligence*.

Daniel Cer, Mona Diab, Eneko Agirre, Inigo Lopez-Gazpio, and Lucia Specia. 2017. Semeval-2017 task 1: Semantic textual similarity-multilingual and cross-lingual focused evaluation. *arXiv preprint arXiv:1708.00055*.

Ting-Yun Chang and Chi-Jen Lu. 2021. Rethinking why intermediate-task fine-tuning works. In *Findings of the Association for Computational Linguistics: EMNLP 2021*, pages 706–713, Punta Cana, Dominican Republic. Association for Computational Linguistics.

Arslan Chaudhry, Marc'Aurelio Ranzato, Marcus Rohrbach, and Mohamed Elhoseiny. 2019. Efficient lifelong learning with a-GEM. In *International Conference on Learning Representations*.

Leshem Choshen, Elad Venezian, Shachar Don-Yehia, Noam Slonim, and Yoav Katz. 2022. Where to start? analyzing the potential value of intermediate models. *arXiv preprint arXiv:2211.00107*.

Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, Parker Schuh, Kensen Shi, Sasha Tsvyashchenko, Joshua Maynez, Abhishek Rao, Parker Barnes, Yi Tay, Noam Shazeer, Vinodkumar Prabhakaran, Emily Reif, Nan Du, Ben Hutchinson, Reiner Pope, James Bradbury, Jacob Austin, Michael Isard, Guy Gur-Ari, Pengcheng Yin, Toju Duke, Anselm Levskaya, Sanjay Ghemawat, Sunipa Dev, Henryk Michalewski, Xavier Garcia, Vedant Misra, Kevin Robinson, Liam Fedus, Denny Zhou, Daphne Ippolito, David Luan, Hyeontaek Lim, Barret Zoph, Alexander Spiridonov, Ryan Sepassi, David Dohan, Shivani Agrawal, Mark Omernick, Andrew M. Dai, Thanumalayan Sankaranarayana Pillai, Marie Pellat, Aitor Lewkowycz, Erica Moreira, Rewon Child, Oleksandr Polozov, Katherine Lee, Zongwei Zhou, Xuezhi Wang, Brennan Saeta, Mark Diaz, Orhan Firat, Michele Catasta, Jason Wei, Kathy Meier-Hellstern, Douglas Eck, Jeff Dean, Slav Petrov, and Noah Fiedel. 2022. Palm: Scaling language modeling with pathways.

Cyprien de Masson d'Autume, Sebastian Ruder, Lingpeng Kong, and Dani Yogatama. 2019. Episodic memory in lifelong language learning. In *Advances in Neural Information Processing Systems*, volume 32. Curran Associates, Inc.

Aditya Deshpande, Alessandro Achille, Avinash Ravichandran, Hao Li, Luca Zancato, Charless Fowlkes, Rahul Bhotika, Stefano Soatto, and Pietro Perona. 2021. A linearized framework and a new benchmark for model selection for fine-tuning. *arXiv preprint arXiv:2102.00084*.

Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 4171–4186, Minneapolis, Minnesota. Association for Computational Linguistics.

Arthur Douillard, Alexandre Ramé, Guillaume Couairon, and Matthieu Cord. 2022. Dytox: Transformers for continual learning with dynamic token expansion. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*.

Beyza Ermis, Giovanni Zappella, Martin Wistuba, Aditya Rawal, and Cedric Archambeau. 2022. Memory efficient continual learning with transformers. In *Advances in Neural Information Processing Systems*.

Adam Fisch, Tal Schuster, Tommi S. Jaakkola, and Regina Barzilay. 2021. Efficient conformal prediction via cascaded inference with expanded admission.

In *International Conference on Learning Representations*.

Jerome H. Friedman. 2001. Greedy function approximation: A gradient boosting machine. *The Annals of Statistics*, 29(5):1189 – 1232.

Tianyu Gao, Adam Fisch, and Danqi Chen. 2021. Making pre-trained language models better few-shot learners. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 3816–3830, Online. Association for Computational Linguistics.

Jonas Geiping and Tom Goldstein. 2022. Cramming: Training a language model on a single gpu in one day. *arXiv preprint arXiv:2212.14034*.

Suchin Gururangan, Ana Marasović, Swabha Swayamdipta, Kyle Lo, Iz Beltagy, Doug Downey, and Noah A. Smith. 2020. Don't stop pretraining: Adapt language models to domains and tasks. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 8342–8360, Online. Association for Computational Linguistics.

Junxian He, Chunting Zhou, Xuezhe Ma, Taylor Berg-Kirkpatrick, and Graham Neubig. 2022. Towards a unified view of parameter-efficient transfer learning. In *International Conference on Learning Representations*.

Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Tom Hennigan, Eric Noland, Katherine Millican, George van den Driessche, Bogdan Damoc, Aurelia Guy, Simon Osindero, Karen Simonyan, Erich Elsen, Oriol Vinyals, Jack William Rae, and Laurent Sifre. 2022. An empirical analysis of compute-optimal large language model training. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, Bruna Morrone, Quentin de Laroussilhe, Andrea Gesmundo, Mona Attariyan, and Sylvain Gelly. 2019. Parameter-efficient transfer learning for nlp. In *International Conference on Machine Learning*.

Edward J Hu, yelong shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*.

Long-Kai Huang, Junzhou Huang, Yu Rong, Qiang Yang, and Ying Wei. 2022. Frustratingly easy transferability estimation. In *International Conference on Machine Learning*, pages 9201–9225.

Joel Jang, Seonghyeon Ye, Sohee Yang, Joongbo Shin, Janghoon Han, Gyeonghun KIM, Stanley Jungkyu Choi, and Minjoon Seo. 2022. Towards continual knowledge learning of language models. In *International Conference on Learning Representations*.

Zixuan Ke, Yijia Shao, Haowei Lin, Tatsuya Konishi, Gyuhak Kim, and Bing Liu. 2023. Continual learning of language models. In *The Eleventh International Conference on Learning Representations*.

James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A. Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, Demis Hassabis, Claudia Clopath, Dharshan Kumaran, and Raia Hadsell. 2017. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13):3521–3526.

Angeliki Lazaridou, Adhiguna Kuncoro, Elena Gribovskaya, Devang Agrawal, Adam Liska, Tayfun Terzi, Mai Gimenez, Cyprien de Masson d'Autume, Tomas Kocisky, Sebastian Ruder, Dani Yogatama, Kris Cao, Susannah Young, and Phil Blunsom. 2021. Mind the gap: Assessing temporal generalization in neural language models. *arXive preprint arXiv:2102.01951*.

Tao Lei, Junwen Bai, Siddhartha Brahma, Joshua Ainslie, Kenton Lee, Yanqi Zhou, Nan Du, Vincent Y. Zhao, Yuexin Wu, Bo Li, Yu Zhang, and Ming-Wei Chang. 2023. Conditional adapters: Parameter-efficient transfer learning with fast inference. *arXiv preprint arXiv:2304.04947*.

Brian Lester, Rami Al-Rfou, and Noah Constant. 2021. The power of scale for parameter-efficient prompt tuning. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 3045–3059, Online and Punta Cana, Dominican Republic. Association for Computational Linguistics.

Xiang Lisa Li and Percy Liang. 2021. Prefix-tuning: Optimizing continuous prompts for generation. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 4582–4597, Online. Association for Computational Linguistics.

Adam Livska, Tom'avs Kovcisk'y, Elena Gribovskaya, Tayfun Terzi, Eren Sezener, Devang Agrawal, Cyprien de Masson d'Autume, Tim Scholtes, Manzil Zaheer, Susannah Young, Ellen Gilsenan-McMahon, Sophia Austin, Phil Blunsom, and Angeliki Lazaridou. 2022. Streamingqa: A benchmark for adaptation to new knowledge over time in question answering models. In *International Conference on Machine Learning (ICML)*.

Minh-Thang Luong, Quoc V. Le, Ilya Sutskever, Oriol Vinyals, and Lukasz Kaiser. 2016. Multi-task se-

quence to sequence learning. In *4th International Conference on Learning Representations, ICLR*.

Peter Mattson, Christine Cheng, Cody Coleman, Greg Diamos, Paulius Micikevicius, David Patterson, Hanlin Tang, Gu-Yeon Wei, Peter Bailis, Victor Bittorf, David Brooks, Dehao Chen, Debojyoti Dutta, Udit Gupta, Kim Hazelwood, Andrew Hock, Xinyuan Huang, Atsushi Ike, Bill Jia, Daniel Kang, David Kanter, Naveen Kumar, Jeffery Liao, Guokai Ma, Deepak Narayanan, Tayo Oguntebi, Gennady Pekhimenko, Lillian Pentecost, Vijay Janapa Reddi, Taylor Robie, Tom St. John, Tsuguchika Tabaru, Carole-Jean Wu, Lingjie Xu, Masafumi Yamazaki, Cliff Young, and Matei Zaharia. 2020. Mlperf training benchmark. *arXiv preprint arXiv:1910.01500*.

Gaurav Menghani. 2023. Efficient deep learning: A survey on making deep learning models smaller, faster, and better. *ACM Comput. Surv.*, 55(12).

German I. Parisi, Ronald Kemker, Jose L. Part, Christopher Kanan, and Stefan Wermter. 2019. Continual lifelong learning with neural networks: A review. *Neural Networks*, 113:54–71.

Jason Phang, Thibault Févry, and Samuel R. Bowman. 2019. Sentence encoders on stilts: Supplementary training on intermediate labeled-data tasks. *arXiv preprint arXiv:1811.01088*.

Clifton Poth, Jonas Pfeiffer, Andreas Rücklé, and Iryna Gurevych. 2021. What to pre-train on? Efficient intermediate task selection. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 10585–10605, Online and Punta Cana, Dominican Republic. Association for Computational Linguistics.

Yada Pruksachatkun, Jason Phang, Haokun Liu, Phu Mon Htut, Xiaoyi Zhang, Richard Yuanzhe Pang, Clara Vania, Katharina Kann, and Samuel R. Bowman. 2020. Intermediate-task transfer learning with pretrained language models: When and why does it work? In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 5231–5247, Online. Association for Computational Linguistics.

Chengwei Qin and Shafiq Joty. 2022. LFPT5: A unified framework for lifelong few-shot language learning based on prompt tuning of t5. In *International Conference on Learning Representations*.

Yujia Qin, Cheng Qian, Xu Han, Yankai Lin, Huadong Wang, Ruobing Xie, Zhiyuan Liu, Maosong Sun, and Jie Zhou. 2023. Recyclable tuning for continual pre-training. *arXiv preprint arXiv:2305.08702*.

Yujia Qin, Jiajie Zhang, Yankai Lin, Zhiyuan Liu, Peng Li, Maosong Sun, and Jie Zhou. 2022. ELLE: Efficient lifelong pre-training for emerging data. In *Findings of the Association for Computational Linguistics: ACL 2022*, pages 2789–2810, Dublin, Ireland. Association for Computational Linguistics.

Jack W. Rae, Sebastian Borgeaud, Trevor Cai, Katie Millican, Jordan Hoffmann, Francis Song, John Aslanides, Sarah Henderson, Roman Ring, Susannah Young, Eliza Rutherford, Tom Hennigan, Jacob Menick, Albin Cassirer, Richard Powell, George van den Driessche, Lisa Anne Hendricks, Maribeth Rauh, Po-Sen Huang, Amelia Glaese, Johannes Welbl, Sumanth Dathathri, Saffron Huang, Jonathan Uesato, John Mellor, Irina Higgins, Antonia Creswell, Nat McAleese, Amy Wu, Erich Elsen, Siddhant Jayakumar, Elena Buchatskaya, David Budden, Esme Sutherland, Karen Simonyan, Michela Paganini, Laurent Sifre, Lena Martens, Xiang Lorraine Li, Adhiguna Kuncoro, Aida Nematzadeh, Elena Gribovskaya, Domenic Donato, Angeliki Lazaridou, Arthur Mensch, Jean-Baptiste Lespiau, Maria Tsimpoukelli, Nikolai Grigorev, Doug Fritz, Thibault Sottiaux, Mantas Pajarskas, Toby Pohlen, Zhitao Gong, Daniel Toyama, Cyprien de Masson d'Autume, Yujia Li, Tayfun Terzi, Vladimir Mikulik, Igor Babuschkin, Aidan Clark, Diego de Las Casas, Aurelia Guy, Chris Jones, James Bradbury, Matthew Johnson, Blake Hechtman, Laura Weidinger, Iason Gabriel, William Isaac, Ed Lockhart, Simon Osindero, Laura Rimell, Chris Dyer, Oriol Vinyals, Kareem Ayoub, Jeff Stanway, Lorrayne Bennett, Demis Hassabis, Koray Kavukcuoglu, and Geoffrey Irving. 2021. Scaling language models: Methods, analysis and insights from training gopher.

Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. 2020. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 21(140):1–67.

Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. 2016. SQuAD: 100,000+ questions for machine comprehension of text. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, pages 2383–2392, Austin, Texas. Association for Computational Linguistics.

Anastasia Razdaibiedina, Yuning Mao, Rui Hou, Madian Khabsa, Mike Lewis, and Amjad Almahairi. 2023. Progressive prompts: Continual learning for language models. In *The Eleventh International Conference on Learning Representations*.

Adam Roberts, Hyung Won Chung, Anselm Levskaya, Gaurav Mishra, James Bradbury, Daniel Andor, Sharan Narang, Brian Lester, Colin Gaffney, Afroz Mohiuddin, Curtis Hawthorne, Aitor Lewkowycz, Alexandru Salcianu, Marc van Zee, Jacob Austin, Sebastian Goodman, Livio Baldini Soares, Haitang Hu, Sasha Tsvyashchenko, Aakanksha Chowdhery, Jasmijn Bastings, Jannis Bulian, Xavier García, Jianmo Ni, Andrew Chen, Kathleen Kenealy, J. Clark, Stephan Lee, Daniel H Garrette, James Lee-Thorp, Colin Raffel, Noam M. Shazeer, Marvin Ritter, Maarten Bosma, Alexandre Passos, Jeremy B. Maitin-Shepard, Noah Fiedel, Mark Omernick, Brennan Saeta, Ryan Sepassi, Alexander Spiridonov,

Joshua Newlan, and Andrea Gesmundo. 2022. Scaling up models and data with t5x and seqio. *ArXiv preprint: arXiv 2203.17189*.

Sebastian Ruder and Barbara Plank. 2017. Learning to select data for transfer learning with Bayesian optimization. In *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing*, pages 372–382, Copenhagen, Denmark. Association for Computational Linguistics.

David Saxton, Edward Grefenstette, Felix Hill, and Pushmeet Kohli. 2019. Analysing mathematical reasoning abilities of neural models. In *International Conference on Learning Representations*.

John Schulman, Barret Zoph, Christina Kim, Jacob Hilton, Jacob Menick, Jiayi Weng, Juan Felipe Ceron Uribe, Liam Fedus, Luke Metz, Michael Pokorny, Rapha Gontijo Lopes, Shengjia Zhao, Arun Vijayvergiya, Eric Sigler, Adam Perelman, Chelsea Voss, Mike Heaton, Joel Parish, Dave Cummings, Rajeev Nayak, Valerie Balcom, David Schnurr, Tomer Kaftan, Chris Hallacy, Nicholas Turley, Noah Deutsch, Vik Goel, Jonathan Ward, Aris Konstantinidis, Wojciech Zaremba, Long Ouyang, Leonard Bogdonoff, Joshua Gross, David Medina, Sarah Yoo, Teddy Lee, Ryan Lowe, Dan Mossing, Joost Huizinga, Roger Jiang, Carroll Wainwright, Diogo Almeida, Steph Lin, Marvin Zhang, Kai Xiao, Katarina Slama, Steven Bills, Alex Gray, Jan Leike, Jakub Pachocki, Phil Tillet, Shantanu Jain, Greg Brockman, Nick Ryder, Alex Paino, Qiming Yuan, Clemens Winter, Ben Wang, Mo Bavarian, Igor Babuschkin, Szymon Sidor, Ingmar Kanitscheider, Mikhail Pavlov, Matthias Plappert, Nik Tezak, Heewoo Jun, William Zhuk, Vitchyr Pong, Lukasz Kaiser, Jerry Tworek, Andrew Carr, Lilian Weng, Sandhini Agarwal, Karl Cobbe, Vineet Kosaraju, Alethea Power, Stanislas Polu, Jesse Han, Raul Puri, Shawn Jain, Benjamin Chess, Christian Gibson, Oleg Boiko, Emy Parparita, Amin Tootoonchian, Kyle Kosic, and Christopher Hesse. 2023. Chatgpt. https://openai.com/blog/chatgpt/.

Thomas Scialom, Tuhin Chakrabarty, and Smaranda Muresan. 2022. Fine-tuned language models are continual learners. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pages 6107–6122, Abu Dhabi, United Arab Emirates. Association for Computational Linguistics.

Tom Veniat, Ludovic Denoyer, and MarcAurelio Ranzato. 2021. Efficient continual learning with modular networks and task-driven priors. In *International Conference on Learning Representations (ICLR)*.

Tu Vu, Tong Wang, Tsendsuren Munkhdalai, Alessandro Sordoni, Adam Trischler, Andrew Mattarella-Micke, Subhransu Maji, and Mohit Iyyer. 2020. Exploring and predicting transferability across NLP tasks. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 7882–7926, Online. Association for Computational Linguistics.

Alex Wang, Jan Hula, Patrick Xia, Raghavendra Pappagari, R. Thomas McCoy, Roma Patel, Najoung Kim, Ian Tenney, Yinghui Huang, Katherin Yu, Shuning Jin, Berlin Chen, Benjamin Van Durme, Edouard Grave, Ellie Pavlick, and Samuel R. Bowman. 2019. Can you tell me how to get past sesame street? sentence-level pretraining beyond language modeling. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 4465–4476, Florence, Italy. Association for Computational Linguistics.

Jason Wei, Maarten Bosma, Vincent Zhao, Kelvin Guu, Adams Wei Yu, Brian Lester, Nan Du, Andrew M. Dai, and Quoc V Le. 2022. Finetuned language models are zero-shot learners. In *International Conference on Learning Representations*.

Maciej Wołczyk, Michał Zając, Razvan Pascanu, Łukasz Kuciński, and Piotr Miłoś. 2021. Continual world: A robotic benchmark for continual reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Tongtong Wu, Massimo Caccia, Zhuang Li, Yuan-Fang Li, Guilin Qi, and Gholamreza Haffari. 2022. Pretrained language model in continual learning: A comparative study. In *International Conference on Learning Representations*.

Kaichao You, Yong Liu, Jianmin Wang, and Mingsheng Long. 2021. Logme: Practical assessment of pre-trained models for transfer learning. In *ICML*.

Kaichao You, Yong Liu, Ziyang Zhang, Jianmin Wang, Michael I. Jordan, and Mingsheng Long. 2022. Ranking and tuning pre-trained models: A new paradigm for exploiting model hubs. *JMLR*.

A Benchmark details

Statistics for individual datasets used in our analysis are contained in Table A.1. Following Wei et al. (2022), we cap the number of training examples per dataset at $30k$, and use up to 200 examples for validation. We use the full test sets where available. Note that some tasks have multiple subtasks (e.g., for SQuAD (Rajpurkar et al., 2016) we treat SQuAD v1 and v2 as separate subtasks). We apply these limits only per *subtask*, and when computing results, we take the average performance across all subtasks. The benchmark is available for download at https://ct5x.s3.us-east-2.amazonaws.com/benchmark.json. All models were trained using the T5x framework (Roberts et al., 2022) using the base T5 1.1 architecture with default hyper-parameters. Our pre-trained model was the `t5_1_1_lm100k_base` model released by Lester et al. (2021).

Name	Train	Validation	Test	Metric
anli	76946	600	3200	accuracy
squad	60000	400	22443	f1
mnli	60000	400	19647	accuracy
hellaswag	30000	200	10042	accuracy
yelp_polarity_reviews	30000	200	38000	accuracy
record	30000	200	10000	accuracy
true_case	30000	200	3000	mean_edit
xsum	30000	200	11301	rougeLsum
newsroom	30000	200	108862	rougeLsum
multi_news	30000	200	5622	rougeLsum
web_nlg_en	30000	200	1667	rougeLsum
qnli	30000	200	5463	accuracy
winogrande	30000	200	1267	accuracy
sentiment140	30000	200	498	accuracy
dart	30000	200	2768	rougeLsum
wiki_lingua_english_en	30000	200	28614	rougeLsum
common_gen	30000	200	993	rougeLsum
word_segment	30000	200	3000	mean_edit
cnn_dailymail	30000	200	11490	rougeLsum
e2e_nlg	30000	200	4299	rougeLsum
glue_qqp	30000	200	40430	accuracy
paws_wiki	30000	200	8000	accuracy
sst2	30000	200	872	accuracy
snli	30000	200	10000	accuracy
trivia_qa	30000	200	11313	f1
ag_news_subset	30000	200	7600	accuracy
quac	30000	200	7354	nll
math_dataset	30000	200	10000	accuracy
drop	30000	200	9536	f1
fix_punct	30000	200	3000	mean_edit
natural_questions	30000	200	3610	f1
gigaword	30000	200	1951	rougeLsum
multirc	27043	200	4848	accuracy
cosmos_qa	25062	200	2985	accuracy
imdb_reviews	24800	200	25000	accuracy
piqa	16013	100	1838	accuracy
samsum	14732	200	819	rougeLsum
aeslc	14436	200	1906	rougeLsum
bool_q	9227	200	3270	accuracy
cola	8351	200	1043	accuracy
coqa	7099	100	500	nll
stsb	5649	100	1500	accuracy
trec	5252	200	500	accuracy
wic	5228	200	638	accuracy
openbookqa	4957	200	500	accuracy
opinion_abstracts	4790	200	1000	rougeLsum
glue_mrpc	3468	200	408	accuracy
arc	2970	400	3548	accuracy
rte	2290	200	277	accuracy
story_cloze	1671	200	1871	accuracy
definite_pronoun_resolution	1122	200	564	accuracy
wnli	605	30	71	accuracy
wsc	504	50	104	accuracy
copa	350	50	100	accuracy
cb	200	50	56	accuracy

Table A.1: Statistics for each dataset used in our analysis, and the primary metric used.

B Checkpoint selection features

We derive a number of easy to compute features for our lightweight GBDT checkpoint selecto when evaluating transfer candidate (t_i, t_j).

- **Relative performance.** We compute the relative 0-shot and 5-shot performance for an independently fine-tuned model (starting from the base model, trained just on t_j) with the sequential tuning candidate (starting from t_i and training next on t_j). Here k-shot denotes the performance after k updates (using a fixed batch-size). We compute the relative performance both in terms of the specific task metric (e.g., accuracy) and the token negative log-likelihood, as these can have different trends.

- **Weight change.** We compute the maximum and average magnitude parameter update of the checkpoint for t_j relative to the base pre-trained model. This is stratified by weight group, where we differentiate between softmax, embedding, and layer parameters (the layers are collapsed into 4 groups by layer number, e.g., layers $\{1, 2, 3\}$ are a group).

- **Update similarity.** We approximate gradient-based similarity by using the weight change of a candidate model from the base pre-trained model as an estimate of the average gradient applied (recall that all models start from the same pre-trained model). We then compare this "average gradient" to the weight change of the independent fine-tuning model after 5 steps using cosine similarity. This gives an idea if the average gradient already applied to t_i is similar in direction to the initial gradient computed when training directly on t_j. We also stratify this metric across different parameter groups (the same as for absolute weight change above).

- **Task metadata.** We include binary features that indicate if the last task used to train t_i is in the same manually defined family as t_j, and also if any task used in the full process for training t_i (i.e., was used as a previous checkpoint initialization) is in the same manually defined family as t_j. The task families are illustrated in Figure 2.

When comparing feature importance determined after training the GBDT, the relative 0 and 5-shot performance is most important, followed by gradient similarity. While this is intuitive (and also clearly helpful for our specific purpose when considering that our log-scale PerAUC metric heavily weights strong early performance), it is important to note that computing these features does not scale particularly well with the number of tasks, as it involves training and evaluating many models, even if only for a few updates. Gradient similarity is less costly to evaluate using our approximate method, as it only requires evaluating the weight change for all checkpoints except for the pre-trained model.

C Additional results

We list all results per sequence type in Tables C.1 through C.10, in addition to the medians in Table 2.

$\mathbf{A} \rightarrow \mathbf{C}$ Pairwise	$\mathbf{B} \rightarrow \mathbf{C}$ Pairwise	$\mathbf{A} \rightarrow \mathbf{B} \rightarrow \mathbf{C}$ Naïve	Selective	Oracle
43.31	22.52	56.97	43.31	56.97
22.52	47.14	54.55	47.14	54.55
49.27	50.82	58.54	58.54	58.54
47.14	50.82	56.69	56.69	56.69
32.59	32.80	31.97	32.59	32.80
37.56	56.60	58.50	56.60	58.50
32.59	59.31	52.37	52.37	59.31
59.31	32.80	56.58	59.31	59.31
22.71	30.83	38.42	22.71	38.42
30.58	28.64	39.13	39.13	39.13
22.71	30.58	40.50	40.50	40.50
36.50	30.58	41.57	36.50	41.57
14.07	12.22	1.37	12.22	14.07
43.37	14.07	43.77	43.37	43.77
12.22	43.37	32.90	12.22	43.37
43.37	17.00	41.58	41.58	43.37

Table C.1: Both **A** and **B** are intended to be positive.

$\mathbf{A} \rightarrow \mathbf{C}$ Pairwise	$\mathbf{B} \rightarrow \mathbf{C}$ Pairwise	$\mathbf{A} \rightarrow \mathbf{B} \rightarrow \mathbf{C}$ Naïve	Selective	Oracle
52.36	-310.39	-291.80	52.36	52.36
51.16	-310.39	-281.67	51.16	51.16
52.36	-17.99	49.96	52.36	52.36
51.16	-57.86	-14.64	51.16	51.16
8.20	-16.31	-8.35	8.20	8.20
13.66	-16.31	7.90	13.66	13.66
11.41	-16.31	10.13	11.41	11.41
14.85	-16.31	-4.11	14.85	14.85
14.07	-17.00	-1.44	0.00	14.07
43.37	-17.00	26.98	43.37	43.37
12.22	-17.00	-3.25	12.22	12.22
17.00	-17.00	-8.93	0.00	17.00
10.99	-18.00	-6.10	10.99	10.99
10.99	-9.45	0.63	10.99	10.99
10.99	-12.91	1.38	10.99	10.99
10.99	-13.55	4.33	10.99	10.99

Table C.2: **A** is intended to be positive while **B** is intended to be negative.

$\mathbf{A} \rightarrow \mathbf{C}$ Pairwise	$\mathbf{B} \rightarrow \mathbf{C}$ Pairwise	$\mathbf{A} \rightarrow \mathbf{B} \rightarrow \mathbf{C}$ Naïve	Selective	Oracle
52.20	-0.78	45.69	52.20	52.20
15.00	2.99	34.87	15.00	34.87
15.00	-3.68	22.51	15.00	22.51
28.70	2.99	23.93	28.70	28.70
17.33	-3.31	10.29	17.33	17.33
14.14	-3.31	-1.38	14.14	14.14
14.14	-2.10	11.34	14.14	14.14
15.27	-3.31	-0.78	15.27	15.27
59.31	0.49	43.87	59.31	59.31
32.80	0.49	27.43	32.80	32.80
59.31	5.51	44.56	59.31	59.31
35.85	4.09	32.40	35.85	35.85
-0.95	0.00	6.95	-0.95	6.95
-0.95	4.09	6.95	-0.95	6.95
-0.95	0.00	-4.45	-0.95	0.00
-0.95	6.95	6.95	6.95	6.95

Table C.3: **A** is intended to be positive while **B** is intended to be neutral.

$\mathbf{A} \rightarrow \mathbf{C}$ Pairwise	$\mathbf{B} \rightarrow \mathbf{C}$ Pairwise	$\mathbf{A} \rightarrow \mathbf{B} \rightarrow \mathbf{C}$ Naïve	Selective	Oracle
-16.31	8.20	-12.27	0.00	8.20
-16.31	13.66	15.92	13.66	15.92
-16.31	14.85	15.13	14.85	15.13
-16.31	11.41	8.87	11.41	11.41
-12.91	10.99	1.06	10.99	10.99
-9.45	10.99	-2.25	10.99	10.99
-13.55	10.99	-2.80	10.99	10.99
-18.00	10.99	-14.24	10.99	10.99
-17.00	17.00	5.67	0.00	17.00
-17.00	43.37	30.98	43.37	43.37
-17.00	12.22	-6.83	12.22	12.22
-17.00	14.07	18.22	0.00	18.22
-310.34	51.16	39.47	51.16	51.16
-57.86	51.16	38.09	51.16	51.16
-17.99	52.36	33.78	33.78	52.36
-17.99	51.16	49.56	49.56	51.16

Table C.4: **A** is intended to be negative while **B** is intended to be positive.

$\mathbf{A} \rightarrow \mathbf{C}$ Pairwise	$\mathbf{B} \rightarrow \mathbf{C}$ Pairwise	$\mathbf{A} \rightarrow \mathbf{B} \rightarrow \mathbf{C}$ Naïve	Selective	Oracle
-17.99	-310.34	-8.89	0.00	0.00
-310.34	-57.86	-66.16	0.00	0.00
-17.99	-310.39	-119.44	0.00	0.00
-57.86	-17.99	-49.32	0.00	0.00
-106.80	-36.99	-40.34	0.00	0.00
-157.10	-66.13	-372.89	0.00	0.00
-106.80	-5.16	-85.35	0.00	0.00
-66.13	-5.16	-22.69	0.00	0.00
-13.55	-18.00	-20.60	0.00	0.00
-9.45	-12.91	-7.13	0.00	0.00
-9.45	-18.00	-5.68	0.00	0.00
-12.91	-18.00	-5.48	0.00	0.00
-16.89	-21.00	-18.95	0.00	0.00
-27.96	-16.89	-22.73	0.00	0.00
-27.96	-21.00	-26.68	0.00	0.00
-21.00	-27.96	-32.79	0.00	0.00

Table C.5

Table C.6: Both **A** and **B** are intended to be negative.

$\mathbf{A} \rightarrow \mathbf{C}$ Pairwise	$\mathbf{B} \rightarrow \mathbf{C}$ Pairwise	$\mathbf{A} \rightarrow \mathbf{B} \rightarrow \mathbf{C}$ Naïve	Selective	Oracle
-17.00	-12.78	-17.39	-12.78	0.00
-17.00	-20.57	-16.32	-20.57	0.00
-17.00	-2.10	-22.33	0.00	0.00
-17.00	2.22	7.99	0.00	7.99
-106.80	5.21	1.10	5.21	5.21
-77.67	-0.73	-10.93	0.00	0.00
-77.67	8.12	1.48	0.00	8.12
-66.13	-0.73	-3.65	0.00	0.00
-16.89	-2.96	-14.71	0.00	0.00
-27.96	-3.43	-15.41	0.00	0.00
-21.00	-4.53	-22.14	0.00	0.00
-27.96	-4.53	-14.35	0.00	0.00
-6.33	-2.75	-2.77	0.00	0.00
-6.33	2.81	-5.74	0.00	2.81
-6.33	-0.97	-4.85	0.00	0.00
-6.33	-1.21	-3.65	0.00	0.00

Table C.7: **A** is intended to be negative while **B** is intended to be neutral.

$\mathbf{A} \rightarrow \mathbf{C}$ Pairwise	$\mathbf{B} \rightarrow \mathbf{C}$ Pairwise	$\mathbf{A} \rightarrow \mathbf{B} \rightarrow \mathbf{C}$ Naïve	Selective	Oracle
-0.70	11.41	8.71	11.41	11.41
-12.99	13.66	10.29	13.66	13.66
2.67	13.66	10.06	13.66	13.66
-0.70	8.20	6.25	8.20	8.20
0.16	16.38	8.50	16.38	16.38
-7.90	7.83	-0.79	-7.90	7.83
0.16	10.45	11.52	10.45	11.52
2.71	10.45	1.90	1.90	10.45
-40.79	52.36	51.84	51.84	52.36
27.80	52.36	44.12	52.36	52.36
-40.79	51.16	48.45	51.16	51.16
27.80	51.16	47.42	51.16	51.16
13.47	34.11	36.22	34.11	36.22
-6.09	45.62	54.49	45.62	54.49
-9.01	80.13	76.83	80.13	80.13
-13.75	57.81	28.08	0.00	57.81

Table C.8

Table C.9: **A** is intended to be neutral while **B** is intended to be positive.

$\mathbf{A} \rightarrow \mathbf{C}$ Pairwise	$\mathbf{B} \rightarrow \mathbf{C}$ Pairwise	$\mathbf{A} \rightarrow \mathbf{B} \rightarrow \mathbf{C}$ Naïve	Selective	Oracle
2.81	-6.33	-1.21	0.00	2.81
-0.97	-6.33	-3.13	-0.97	0.00
-3.74	-6.33	-4.13	0.00	0.00
-3.64	-6.33	-2.45	0.00	0.00
-2.96	-21.00	-22.89	0.00	0.00
-2.96	-16.89	-12.74	0.00	0.00
-4.53	-16.89	-14.76	0.00	0.00
-3.32	-16.89	-17.37	0.00	0.00
8.12	-5.16	1.27	0.00	8.12
8.12	-5.75	3.54	0.00	8.12
5.21	-66.13	-9.05	5.21	5.21
5.21	-36.99	-33.14	5.21	5.21
-1.66	-17.00	-12.37	-1.66	0.00
-20.57	-17.00	-25.37	-20.57	0.00
-2.10	-17.00	-14.88	0.00	0.00
1.28	-17.00	-21.27	0.00	1.28

Table C.10: **A** is intended to be neutral while **B** is intended to be negative.